

Wstęp do uczenia maszynowego - projekt

Przemysław Kania
Numer indeksu: 333063
Grupa: GR 3

21 grudnia 2025

1 Cel projektu

Celem projektu jest zaproponowanie metody klasyfikacji, która pozwoli zbudować model o jak największej mocy predykcyjnej. Dysponujemy sztucznie wygenerowanym zbiorem danych *artificial*, w którym zostały ukryte istotne zmienne. Należy dokonać klasyfikacji do dwóch klas. Dokładność modelu będzie mierzona za pomocą miary zrównoważonej dokładności (*balanced accuracy*).

2 Analiza zadania i danych wejściowych

Podczas badań posługujemy się zbiorem danych *artificial*. Udostępniono go w formie podzielonej na 3 podzbiory, dwa treningowe oraz jeden testowy. Są to następujące pliki, które zostaną wykorzystane do analizy:

- zbiór treningowy: **artificial_train_data.csv**,
- etykiety zbioru treningowego: **artificial_train_labels.csv**,
- zbiór testowy: **artificial_test_data.csv**.

Główne pakiety z biblioteki Pythona użyte do pracy z danymi to *pandas* oraz *numpy*. Przytoczony wcześniej zbiór *artificial* zbiór danych jest sztucznie wygenerowanym zbiorem, który zawiera 100 zmiennych objaśniających (kolumn). Zbiór treningowy zawiera 1500 obserwacji, natomiast testowy zawiera ich 500. Etykiety to 1500 wierszy oraz 1 kolumna, przyporządkowane wierszowo zbiorowi treningowemu. Ten składa się z cyfr 1 oraz 2 w liczbie odpowiednio 755 i 745, a ich stosunek do całości to 0,503 i 0,497. Można więc stwierdzić, że są one zbalansowane. Warto zaznaczyć także, że wszystkie trzy podzbiory nie zawierają braków danych, a więc nie ma potrzeby ich ewentualnego uzupełniania w dalszej części projektu.

3 Obróbka danych wejściowych

Kolejny krok w rozwiązaniu zadania bazuje na *Pipeline*. Można go interpretować jako pewnego rodzaju linię produkcyjną w fabryce. Pozwala on na sprawne połączenie całości procesu trenowania w jeden uczący się algorytm. Kryje się pod tym jednocześnie obróbka danych wejściowych, aby były jak najbardziej poprawne i czytelne dla późniejszych kroków, a także automatyczne ich wykorzystanie w końcowej fazie do wyznaczania predykcji.

W *Pipeline* stworzonym na potrzeby zadania, zawartych zostało kilka istotnych kwestii ulepszających dane wejściowe. Na samym początku warto ponownie zaznaczyć, że nie ma potrzeby uzupełniania w jakikolwiek sposób braków danych. Algorytm więc zaczyna od skalowania danych. Następnie wyrzuca kolumny o bardzo małej wariancji, której próg został ustawiony na 0,05. Ciekawostką jest fakt, że testowany był także współczynnik korelacji między poszczególnymi kolumnami. Te, które zbyt mocno korelowały ze sobą wzajemnie w parach były usuwane (do tego celu została stworzona specjalna klasa *Usun_mocno_skorelowane*, której granica dopuszczalnej korelacji ustawiona była na 0.95). Jednak pogorszyło to nieco wyniki końcowe, więc pomysł został porzucony (stworzona klasa znajduje się jednak w skrypcie). Biorąc zatem pod uwagę tylko skalowanie oraz wariancję, dopiero po tych wykonanych przez *Pipeline* czynnościach, udoskonalone dane zostają przepuszczone dalej do algorytmów, które będą się na nich uczyć.

4 Szukanie najlepszego modelu

Po omówieniu jak konkretnie działa część stworzonego *Pipeline'a*, służąca do obróbki danych wejściowych, można przejść do ostatecznej fazy projektu, czyli do znalezienia modelu o jak największej mocy predykcyjnej. Pod lupę zostały wzięte następujące algorytmy poznane na zajęciach przedmiotu Wstęp do Uczenia Maszynowego wraz z narzuconymi im z góry niektórymi hiperparametrami:

- **Decision Tree** (`class_weight="balanced"`, `random_state=1000`),
- **Logistic Regresion** (`class_weight="balanced"`, `max_iter=500`, `random_state=1000`),
- **SVM** (`class_weight="balanced"`, `probability=True`, `random_state=1000`),
- **LDA, QDA**,
- **K-nearest Neighbors**,
- **Naive Bayes**,
- **Bagging Classifier** (`random_state=1000`),
- **Extra Tree** (`class_weight="balanced"`, `random_state=1000`),
- **Gradient Boosting** (`random_state=1000`).

Jednakże aby poszukiwania były jak najbardziej profesjonalne, dla każdego z powyższych algorytmów sprawdzane zostają różne wartości innych hiperparametrów przez nie przyjmowanych (rozważane w rozwiązaniu zestawienie hiperparametrów dla modeli znajduje się na końcu raportu). Całe poszukiwanie najlepszego jest oparte na pętli, która wybiera każdego kandydata z listy dostępnych. Na początku program za pomocą *GridSearchCV* znajduje najlepsze wartości zmiennych z zestawienia dla każdego z poszczególnych modeli, a następnie zbiera je w postaci tabeli. Cały proces selekcji opiera się na macierzy zrównoważonej dokładności - BA (*balanced accuracy*). Ostateczna wersja wspomnianej tabeli zawiera wybrane najlepsze algorytmy, ich hiperparametry oraz wartość miary BA dla każdego z nich. Finalnie aby dowiedzieć się, który skonstruowany model ma faktycznie największą moc predykcyjną dla naszych danych, wybrany zostaje ten o największej wartości miary BA. Są to jednak tylko i wyłącznie przypuszczenia, gdyż nie ma w 100% pewności, że wyznaczony tą drogą model rzeczywiście jest tym najlepszym.

5 Podsumowanie

Modelem o przypuszczalnie największej mocy predykcyjnej dla zbioru *artificial* jest:

- **Extra Tree**;
- `max_features`: **None**, `n_estimators`: **200**;
- **BA = 0.8748656489575197**;

Etykiety (1 i 2) zbioru testowego przewidziane tym najlepszym algorytmem, pojawiają się odpowiednio w liczbie 237 i 263 i stosunku 0,474 i 0,526 do całości. Etykiety, jako tabela z danymi zostały zapisane do pliku *333063_artifical_prediction.txt*.

Tabela 1: Wartości hiperparametrów przeszukiwane w ramach Grid Search

Algorytm	Hiperparametr	Badane wartości
Decision Tree	<code>criterion</code>	{gini, entropy}
	<code>max_depth</code>	{None, 5, 7, 9, 11, 13, 15}
	<code>min_samples_leaf</code>	{3, 5, 7, 9, 11}
Logistic Regression	<code>solver</code>	{liblinear, lbfgs, saga}
	<code>penalty</code>	{l1, l2, elasticnet, None}
	<code>C</code>	{0.001, 0.01, 0.1, 1, 10, 100, 1000}
	<code>l1_ratio</code>	{0.2, 0.5, 0.8} (tylko dla saga + elasticnet)
SVM	<code>kernel</code>	{rbf}
	<code>C</code>	{0.001, 0.01, 0.1, 1, 10, 100}
K-Nearest Neighbors	<code>n_neighbors</code>	{10, 15, 20, 25, 30, 35, 40}
	<code>weights</code>	{distance, uniform}
	<code>metric</code>	{euclidean, manhattan, minkowski}
Naive Bayes	<code>alpha</code>	{0.01, 0.1, 0.5, 1, 10}
	<code>binarize</code>	{-0.5, 0, 0.5}
LDA	<code>solver</code>	{svd}
	<code>tol</code>	{1e-4, 1e-5}
QDA	<code>reg_param</code>	{0, 0.1, 0.5, 0.9}
	<code>tol</code>	{1e-4, 1e-5}
Bagging Classifier	<code>estimator</code>	{DecisionTreeClassifier()}
	<code>n_estimators</code>	{50, 75, 100, 125, 150, 175, 200}
Extra Trees	<code>n_estimators</code>	{50, 100, 150, 200, 250, 300, 350, 400}
	<code>max_features</code>	{None, sqrt}
Gradient Boosting	<code>learning_rate</code>	{0.05, 0.1, 0.2, 0.3, 0.4}
	<code>n_estimators</code>	{50, 100, 150, 200, 250, 300}